Package 'mathmodels'

June 29, 2026

Type Package

Title Comprehensive Mathematical Modeling Algorithms in R

Version 0.0.9

Author Jingxin Zhang

Maintainer Jingxin zhang <zhjx_19@163.com>

Description A versatile R package for mathematical modeling, developed as a companion to Mathematical Modeling: Algorithms and Programming Implementation (China Machine Press). The package implements algorithms across differential and difference equations, statistical analysis, optimization, evaluation, and prediction. Currently, it focuses on evaluation algorithms, including indicator data preprocessing (e.g., standardization, rescaling), subjective and objective weighting methods (e.g., AHP, entropy weighting, CRITIC, PCA weighting) and weight combination, TOPSIS, fuzzy comprehensive evaluation, grey relational analysis, rank sum ratio, DEA (CCR, BCC, SBM, Malmquist, powered by lpSolveAPI), regional economics (LQ, HHI and EG index), inequality measures (e.g., Gini and Theil indices), grey prediction models (e.g., GM(1,1), GM(1,N), Verhulst), Markov chain prediction, and Grey-Markov combined prediction. It also provides ordinary differential equation (ODE) solvers for compartmental epidemic models (SIR, SEIR, SEIRS, SIDP), along with epidemic visualization and summary metrics. Designed for researchers and practitioners in mathematical modeling.

License AGPL (>= 3)

URL https://github.com/zhjx19/mathmodels, https://zhjx19.github.io/mathmodels/

BugReports https://github.com/zhjx19/mathmodels/issues

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

Imports lpSolveAPI,
dplyr (>= 1.0.0),
forecast,
ggplot2 (>= 3.5.0),
MASS,
purrr,
readxl,
rlang,
stats,
stringr,
tibble,
tidyr (>= 1.1.0),

tseries,
deSolve,
patchwork,
rugarch

Depends R (>= 4.1)

Suggests knitr,
quarto,
rmarkdown,
scales,
testthat (>= 3.0.0)

VignetteBuilder knitr

Config/testthat/edition 3

Config/roxygen2/version 8.0.0

Contents

AHP *AHP: Analytic Hierarchy Process*

Description

AHP is a multi-criteria decision analysis method developed by Saaty, which can also be used to determine indicator weights.

Usage

```
AHP(A)
```

Arguments

A	a numeric matrix, i.e. pairwise comparison matrix

Value

a list object that contains: w (Weight vector), CR (Consistency ratio), Lmax (Maximum eigenvalue), CI (Consistency index)

Examples

```
A = matrix(c(1,   1/2, 4, 3,   3,
             2,   1,   7, 5,   5,
             1/4, 1/7, 1, 1/2, 1/3,
             1/3, 1/5, 2, 1,   1,
             1/3, 1/5, 3, 1,   1), byrow = TRUE, nrow = 5)
AHP(A)
```

combine_preds *Combine Multiple Prediction Results*

Description

Combines multiple prediction results (e.g., from grey prediction, time series, or machine learning models) into a single prediction using a similarity-based weighting approach, improving prediction accuracy.

Usage

```
combine_preds(x)
```

Arguments

x	Numeric vector, prediction results to be combined (length >= 2).

Details

The function combines prediction results by constructing a similarity matrix based on cosine transformation of pairwise differences. Weights are derived from the principal eigenvector of the similarity matrix, ensuring predictions closer to each other have higher influence. For two predictions, equal weights (0.5, 0.5) are used. If all predictions are identical, equal weights are assigned. Compatible with the `mathmodels` package for enhancing prediction models, including grey prediction, time series, or ensemble machine learning.

Value

A list with two elements:

- a: Numeric, the combined prediction value.
- w: Numeric vector, weights for each prediction in x, summing to 1.

Examples

```
# Example: Combine three prediction results
preds = c(100, 102, 98)  # E.g., from grey prediction, ARIMA, or ML models
combine_preds(preds)
```

combine_weights *Combine Subjective and Objective Weights*

Description

Combines subjective and objective weights using linear, multiplicative, or game theory-based methods (geometric mean or linear system).

Usage

```
combine_weights(w_subj, w_obj, type = "linear", alpha = 0.5)
```

Arguments

w_subj	Numeric vector of subjective weights.
w_obj	Numeric vector of objective weights.
type	Character string specifying the combination method: "linear", "multiplicative", "game", or "game_linear".
alpha	Numeric value between 0 and 1, used only for the linear method to weight subjective weights. Defaults to 0.5.

Details

The function supports four methods:

- Linear: Combines weights as alpha * w_subj + (1 - alpha) * w_obj.
- Multiplicative: Combines weights as w_subj * w_obj, requiring positive weights.
- Game: Uses the geometric mean (sqrt(w_subj * w_obj)) to balance weights.
- Game_linear: Uses a game-theoretic approach by solving a linear system based on the cross-product of weights.

Value

A numeric vector of combined weights, normalized to sum to 1.

Examples

```
w_subj = c(0.4, 0.3, 0.2, 0.1)
w_obj = c(0.25, 0.2, 0.3, 0.25)
combine_weights(w_subj, w_obj, type = "linear", alpha = 0.6)
combine_weights(w_subj, w_obj, type = "multiplicative")
combine_weights(w_subj, w_obj, type = "game")
combine_weights(w_subj, w_obj, type = "game_linear")
```

---

compute_mf *Compute fuzzy membership vector and return corresponding membership functions.*

Description

compute_mf transforms a single indicator value into a fuzzy membership vector, where each element represents the degree of membership to a specific evaluation level. compute_mf_funs returns the list of membership functions for visualization purposes.

Usage

```
compute_mf_funs(thresholds)

compute_mf(x, thresholds)
```

Arguments

thresholds	A numeric vector containing at least two threshold values that define the boundaries between evaluation levels.
x	A numeric scalar representing the value of an indicator.

Value

A list with two elements:

mv A numeric vector, membership degrees to each level.

mfs A list of functions, one per level, for plotting membership functions.

Examples

```
# Example: SO2 concentration = 0.07, thresholds = c(0.05, 0.15, 0.25, 0.5)
th = c(0.05, 0.15, 0.25, 0.5)
compute_mf(0.07, th)

## Not run:
mfs = compute_mf_funs(th)
plots = lapply(mfs, \(x) plot_mf(x, xlim = c(0, 0.6)))
gridExtra::grid.arrange(grobs = plots, nrow = 2)

## End(Not run)
```

---

critic_weight *CRITIC Weight Method*

Description

Computes objective weights of indicators and scores of samples using the CRITIC method. The method considers both the contrast intensity (e.g., standard deviation or entropy) and conflict among indicators (based on correlation) to determine indicator importance. This version supports different methods for contrast intensity and correlation types.

Usage

```
critic_weight(
  X,
  index = NULL,
  method = "std",
  cor_method = "pearson",
  epsilon = 0.002
)
```

Arguments

X	A numeric data frame or matrix where rows represent samples (observations) and columns represent indicators (variables).
index	A character vector indicating the direction of each indicator: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already normalized indicators (no rescaling will be applied).
	If `index = NULL` (default), all indicators are treated as `NA`, meaning no nor
method	Character scalar; specifies the method used to compute contrast intensity. Options: "std" (standard deviation, default), or "entropy" (based on information redundancy).
cor_method	Character scalar; specifies the method for computing correlations. Options: "pearson" (default), "spearman", or "kendall".
epsilon	A small constant used to replace exact 0s and 1s in the data to prevent log(0) errors. Default is 0.002. Only used when method = "entropy".

Value

A list containing:

w	Numeric vector of weights for each indicator.
s	Numeric vector of scores for each sample (row), scaled by 100.

Examples

```
# Example: Using CRITIC method on a simple dataset
X = data.frame(
  x1 = c(3, 5, 2, 7),
  x2 = c(10, 20, 15, 25)
)
index = c("+", "-")
critic_weight(X, index)
critic_weight(X, index, method = "entropy")
```

---

cv_weight *Coefficient of Variation Weighting*

Description

Computes weights for indicators using the Coefficient of Variation (CV) method. Weights are derived by normalizing the CV (standard deviation divided by mean) for each indicator.

Usage

```
cv_weight(X)
```

Arguments

X	Numeric matrix or data frame with positive indicator data.

Details

The cv_weight function calculates weights using the CV method. For each column in data, the CV is computed as the standard deviation divided by the mean. Weights are obtained by normalizing the CVs to sum to 1. This lightweight implementation uses base R and assumes all columns are numeric indicators.

Value

Numeric vector of weights for the indicators, summing to 1.

Examples

```
X = data.frame(x1 = c(10, 20, 15), x2 = c(5, 10, 8))
cv_weight(X)
```

DEA *DEA efficiency analysis*

Description

DEA efficiency analysis

Usage

```
basic_DEA(
  data,
  inputs,
  outputs,
  ud_outputs = NULL,
  orientation = "io",
  rts = "vrs"
)

super_DEA(data, inputs, outputs, orientation = "io", rts = "vrs")

basic_SBM(
  data,
  inputs,
  outputs,
  ud_outputs = NULL,
  orientation = "io",
  rts = "vrs"
)

super_SBM(data, inputs, outputs, orientation = "io", rts = "vrs")

malmquist(
  data,
  period,
  inputs,
```

```
  outputs,
  orientation = "oo",
  rts = "vrs",
  type1 = "glob",
  type2 = "rd"
)
```

Arguments

data	A data frame. 1st column = DMU names.
inputs	Column indices/names of input variables.
outputs	Column indices/names of output variables.
ud_outputs	Column indices (within outputs) of undesirable outputs, or NULL.
orientation	"io" (input-oriented, default) or "oo" (output-oriented). All DEA functions return Farrell efficiency (0~1), where 1 = efficient. For output orientation: value = $1/\phi$, where $\phi \geq 1$ is the Shephard distance.
rts	"vrs" (variable returns to scale, default) or "crs" (constant).
period	Column name or index identifying the time period variable.
type1	"cont" (contemporaneous), "seq" (sequential), or "glob" (global, default).
type2	"fgnz" (Färe-Grosskopf-Norris-Zhang) or "rd" (Ray-Desli, default).

Value

A list with components: efficiencies, lambdas, slacks, targets, returns, model, orientation, dmu.

Functions

- basic_DEA(): Standard radial DEA (CCR/BCC)
- super_DEA(): Super-efficiency DEA (self excluded, radial only)
- basic_SBM(): Standard Slacks-Based Measure (SBM, Tone 2001)
- super_SBM(): Super-efficiency SBM (self excluded, no undesirable outputs)
- malmquist(): Malmquist productivity index

Examples

```
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
basic_DEA(df, inputs = 2:3, outputs = 4, rts = "crs")
basic_DEA(df, inputs = 2:3, outputs = 4, rts = "vrs")
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
super_DEA(df, inputs = 2:3, outputs = 4, rts = "crs")
```

```
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
basic_SBM(df, inputs = 2:3, outputs = 4, rts = "crs")
df = data.frame(
  DMU = paste0("DMU", 1:7),
  x1  = c(20, 60, 40, 60, 70, 30, 50),
  x2  = c(151, 200, 120, 170, 250, 210, 90),
  y1  = c(100, 210, 150, 240, 220, 80, 200)
)
super_SBM(df, inputs = 2:3, outputs = 4, rts = "crs")
panel = data.frame(
  DMU    = rep(paste0("DMU", 1:5), 3),
  Period = rep(1:3, each = 5),
  x1     = c(10, 20, 15, 25, 30, 12, 22, 17, 27, 32, 14, 24, 19, 29, 34),
  y1     = c(100, 150, 120, 180, 200, 110, 160, 130, 190, 210, 120, 170, 140, 200, 220)
)
malmquist(panel, period = "Period", inputs = 3, outputs = 4,
  rts = "crs", type1 = "cont", type2 = "fgnz")
```

`defuzzify` *Defuzzification Methods for Fuzzy Comprehensive Evaluation*

Description

Implements defuzzification methods for fuzzy evaluation vectors, including weighted average and maximum membership methods.

Usage

```
defuzzify(mu, scores, method = "weighted_average")
```

Arguments

`mu`	Numeric vector, membership degrees for evaluation levels, in `[0, 1]`.
`scores`	Numeric vector, scores corresponding to each evaluation level (e.g., c(100, 80, 60, 40) for "Excellent", "Good", "Fair", "Poor").
`method`	Character, defuzzification method: "weighted_average", "max_membership", "centroid".

Value

Numeric, defuzzified output value.

Examples

```
# Example: Defuzzify fuzzy evaluation vectors for three schemes
mu = c(0.318, 0.351, 0.203, 0.128)
scores = c(30, 60, 75, 90)  # Scores for "Poor", "Fair", "Good", "Excellent"
defuzzify(mu, scores, method = "weighted_average")
defuzzify(mu, scores, method = "max_membership")
defuzzify(mu, scores, method = "centroid")
```

`entropy_weight` *Entropy Weight Method*

Description

Computes the weights of indicators and scores of samples based on the entropy method. This method objectively determines the importance of each indicator according to the amount of information it contains.

Usage

```
entropy_weight(X, index = NULL, epsilon = 0.002)
```

Arguments

`X`	A numeric data frame or matrix where rows represent samples (observations) and columns represent indicators (variables).
`index`	A character vector indicating the direction of each indicator. Use `"+"` for positive indicators (higher is better), `"-"` for negative indicators (lower is better), and `NA` for already normalized indicators (no rescaling will be applied, but minor adjustments will still be made to avoid log(0) errors). If `index = NULL` (default), all indicators are treated as `NA`, meaning no normalization or rescaling is performed, but a small adjustment is still applied to prevent log(0) errors.
`epsilon`	A small constant used to replace exact 0s and 1s in the data to prevent log(0) errors. Default is 0.002.

Value

A list containing:

`w`	Numeric vector of weights for each indicator.
`s`	Numeric vector of scores for each sample (row), scaled by 100.

Examples

```
X = data.frame(
  x1 = c(3, 5, 2, 7),
  x2 = c(10, 20, 15, 25)
)
index = c("+", "-")
entropy_weight(X, index)
```

epi_metrics *Extract key epidemic metrics*

Description

Returns a named list of 4 core scalar metrics derived from ODE output.

Usage

```
epi_metrics(df, beta, gamma, N = NULL)
```

Arguments

df	A data frame with columns time, S, I.
beta	Transmission rate (numeric).
gamma	Recovery / removal rate (numeric).
N	Total population (numeric). If NULL (default), computed as the sum of compartment values at the first time point. attack_rate.

Value

A named list with components: R0 (basic reproduction number), peak_infection (maximum number of infectious individuals), peak_time (time at which peak occurs), attack_rate (proportion of susceptible that became infected).

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
epi_metrics(sir, beta = 0.002, gamma = 0.1)
```

fuzzy_eval *Fuzzy Comprehensive Evaluation*

Description

Performs fuzzy comprehensive evaluation using different fuzzy composition operators to combine factor weights with a fuzzy evaluation matrix. Suitable for multi-criteria decision analysis with weights from methods like AHP, entropy, CRITIC, CV, or PCA.

Usage

```
fuzzy_eval(w, R, type)
```

Arguments

w	Numeric vector, factor weights (e.g., from combine_weights).
R	Numeric matrix, fuzzy evaluation matrix with columns as factors and rows as evaluation grades. Values should be in [0, 1].
type	Integer or character (1-5), specifying the fuzzy composition operator: • 1: Min-max (main factor decisive). • 2: Product-max (main factor prominent). • 3: Weighted sum (additive average). • 4: Bounded sum of mins (min-sum bounded). • 5: Normalized min-sum (balanced average).

Details

The function computes a fuzzy comprehensive evaluation vector B based on the weight vector w and fuzzy evaluation matrix R. Five composition operators are supported:

- Type 1 (min-max): max(min(w, R[j,])), emphasizes the main factor.
- Type 2 (product-max): max(w * R[j,]), highlights the main factor.
- Type 3 (weighted sum): sum(w * R[j,]), additive average.
- Type 4 (bounded sum): min(1, sum(min(w, R[j,]))), bounds the sum of mins.
- Type 5 (normalized min-sum): sum(min(w, R[j,]/sum(R[j,]))), balanced average.

The output B is normalized to sum to 1. If the sum is zero, an error is thrown. Uses base R for lightweight implementation.

Value

A numeric vector of normalized comprehensive evaluation results, summing to 1.

Examples

```
w = c(0.3, 0.3, 0.3, 0.1)  # weights (e.g., from AHP or entropy)

# fuzzy evaluation matrix (3 grades for 4 factors)
R = matrix(c(0.8, 0.7, 0.6, 0.7,
             0.1, 0.2, 0.2, 0.1,
             0.1, 0.1, 0.2, 0.2), nrow = 3, byrow = TRUE)
# Apply fuzzy comprehensive evaluation
fuzzy_eval(w, R, type = 3)  # Weighted sum
```

---

grey_analysis *Grey Relational Analysis Functions*

Description

A collection of functions for performing grey relational analysis, including calculation of grey correlation degree and evaluation based on grey correlation. These functions are designed for decision-making and data analysis by measuring the relational degree between sequences.

Usage

```
grey_corr(ref, cmp, rho = 0.5, w = NULL)

grey_corr_topsis(X, w, index = NULL, rho = 0.5)
```

Arguments

ref	Numeric vector, the reference sequence for grey_corr.
cmp	Numeric matrix or data frame, the comparison sequences for grey_corr.
rho	Numeric scalar, the distinguishing coefficient (default = 0.5).
w	Numeric vector, weights for weighted correlation (default = equal weights).
X	Numeric matrix or data frame, the decision matrix for grey_corr_topsis.
index	Character vector indicating indicator direction: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already rescaled indicators (no rescaling will be applied). If index = NULL (default), all indicators are treated as NA, meaning no rescaling is performed.

Details

These functions implement grey relational analysis for evaluating relationships between sequences or decision alternatives:

grey_corr Computes the grey correlation degree between a reference sequence (ref) and comparison sequences (cmp) using the distinguishing coefficient (rho) and optional weights (w).

grey_corr_topsis Evaluates a decision matrix (X) by normalizing it, applying weights (w), computing grey correlation with the ideal sequence. Direction of indicators can be specified via index.

Value

grey_corr Returns a numeric vector of grey correlation degrees for each comparison sequence.

grey_corr_topsis Returns a numeric vector of relative closeness (grey correlation degrees).

Examples

```
# Grey correlation degree
ref = c(0.9, 0.8, 0.7)
cmp = data.frame(
  x1 = c(0.9, 0.7, 0.8),
  x2 = c(0.8, 0.9, 0.7),
  x3 = c(0.7, 0.8, 0.9)
)
grey_corr(ref, cmp, rho = 0.5)

# Grey correlation evaluation
X = data.frame(x1 = c(8, 7, 6), x2 = c(150, 180, 200), x3 = c(60, 80, 100))
w = c(0.3, 0.4, 0.3)
idx = c("+", "+", "+")
grey_corr_topsis(X, w, idx, rho = 0.5)
```

grey_models *Grey Prediction Models*

Description

Implements grey prediction models for time series forecasting: GM11 applies the GM(1,1) model with level ratio test. GM1N applies the GM(1,N) model with multiple related factors. DGM21 applies the DGM(2,1) model for second-order dynamics. verhulst applies the Verhulst model for logistic growth.

Usage

```
GM11(X)

GM1N(dat, new_data = NULL)

DGM21(X)

verhulst(X)
```

Arguments

X	For GM11, DGM21, verhulst: Numeric vector of original time series data.
dat	For GM1N: Data frame or matrix, last column is characteristic series, others are related factors.
new_data	For GM1N: Optional future values of related factors (1 row, m-1 columns).

Value

For GM11: List with fitted values (fitted), next prediction (pnext), prediction function (f), matrix (mat), parameters (u), level ratios (lambda), and range (rng). For GM1N: List with fitted values (fitted), posterior variance ratio (C), small error probability (P), and prediction function (f). For DGM21, verhulst: List with fitted values (fitted), next prediction (pnext), prediction function (f), matrix (mat), and parameters (u).

Examples

```
# Sample time series for GM11, DGM21, Verhulst
x = c(100, 120, 145, 175, 210)

# GM11
result = GM11(x)
result$fitted    # Fitted values
result$pnext     # Next prediction
result$f(6:8)    # Predict next 3 periods

# DGM21
x = c(2.874,3.278,3.39,3.679,3.77,3.8)
result = DGM21(x)
result$fitted    # Fitted values
result$pnext     # Next prediction
result$f(6:8)    # Predict next 3 periods
```

```
# Verhulst
x = c(4.93,2.33,3.87,4.35,6.63,7.15,5.37,6.39,7.81,8.35)
result = verhulst(x)
result$fitted    # Fitted values
result$pnext     # Next prediction
result$f(6:8)    # Predict next 3 periods

# Sample data for GM1N
data = data.frame(
  factor1 = c(50, 55, 60, 65, 70),
  factor2 = c(20, 22, 25, 28, 30),
  output = c(100, 120, 145, 175, 210)
)
result = GM1N(data)
result$fitted
```

`inequality` *Inequality Indices*

Description

Computes inequality indices for individual or grouped data: `gini0` calculates the Gini coefficient for individual sample data. `gini` calculates the Gini coefficient for grouped data using income and population shares. `theil0` calculates the Theil index for individual sample data. `theil` calculates the Theil index for grouped average data. `theil0_g` calculates the Theil index and decomposition for grouped sample data. `theil_g` calculates the Theil index and decomposition for grouped average data. `theil_g2_cross` calculates the Theil index and decomposition for two-level cross-grouped average data. `theil_g2_nest` calculates the Theil index and decomposition for two-level nested grouped average data.

For `theil`, `theil_g`, `theil_g2_cross`, `theil_g2_nest`: Name of population variable (character).

Usage

```
gini0(y)

gini(y, pop)

theil0(y)

theil(y, pop)

theil0_g(data, group, y)

theil_g(data, group, y, pop)

theil_g2_cross(data, group1, group2, y, pop)

theil_g2_nest(data, group1, group2, y, pop)
```

Arguments

y	For gini0, gini, theil0: Numeric vector of individual incomes.
pop	For gini: Numeric vector of group populations or population shares. For theil, theil0_g, theil_g, theil_g2_cross, theil_g2_nest: Name of income variable (character).
data	For theil0_g, theil_g, theil_g2_cross, theil_g2_nest: Data frame containing variables.
group	For theil0_g, theil_g: Name of grouping variable (e.g., province).
group1	For theil_g2_cross, theil_g2_nest: Name of first grouping variable (e.g., region or province).
group2	For theil_g2_cross, theil_g2_nest: Name of second grouping variable (e.g., type or city).

Value

For gini0, gini: Numeric Gini coefficient (0 to 1). For theil0, theil: Numeric Theil index. For theil0_g, theil_g, theil_g2_cross, theil_g2_nest: List with two vectors:

- theil: theil (Theil index and its decomposition),
- ratio: ratio (contribution rates of each component).

Examples

```
# Sample data
income = c(10, 20, 30, 40, 100)
pop = c(100, 150, 200, 250, 300)

# Gini coefficient (individual data)
gini0(income)

# Gini coefficient (grouped data)
gini(income, pop)

# Theil index (individual sample)
data = data.frame(g = c("A","A",rep("B",10),rep("A",6)),
                  y = c(10,10,rep(8,4),rep(6,6),rep(4,4),2,2))
theil0(data$y)

# Theil index (grouped average)
data2 = data |> dplyr::count(g, y, name = "pop")
theil(data2$y, data2$pop)

# Theil index with grouping (sample data)
theil0_g(data, "g", "y")

# Theil index with grouping (average data)
theil_g(data2, "g", "y", "pop")

# Theil index with two-level cross-grouping
data3 = data.frame(
  industry = c("A", "A", "A", "A", "B", "B", "B", "B"),
  area = c("East", "East", "West", "West", "East", "East", "West", "West"),
  province = c("Shanghai", "Beijing", "Sichuan", "Yunnan",
```

```
              "Shanghai", "Beijing", "Sichuan", "Yunnan"),
  avg_wage = c(500, 400, 80, 60, 300, 250, 50, 40),
  emp_num = c(100, 80, 90, 70, 120, 100, 80, 60)
)
theil_g2_cross(data3, "industry", "area", "avg_wage", "emp_num")

# Theil index with two-level nested grouping
data4 = data.frame(
  province = c("A", "A", "A", "A", "B", "B"),
  city = c("A1", "A1", "A2", "A2", "B1", "B1"),
  industry = c("Manu", "Serv", "Manu", "Serv", "Manu", "Serv"),
  y = c(50000, 45000, 60000, 55000, 70000, 65000),
  pop = c(10000, 8000, 15000, 12000, 10000, 8000)
)
theil_g2_nest(data4, "province", "city", "y", "pop")
```

`linear_sum` *Linear weighted synthesis*

Description

Linear weighted synthesis

Usage

```
linear_sum(data, w)
```

Arguments

`data`	Indicator matrix or data frame (rows = samples, columns = indicators)
`w`	Weight vector (length must match ncol(data))

Value

Vector of comprehensive scores

Examples

```
data = data.frame(
  GDP = c(0.85, 0.72, 0.91, NA),
  Employment = c(0.78, 0.85, 0.67, 0.73),
  Environment = c(0.65, 0.72, NA, 0.81)
)
w = c(0.5, 0.3, 0.2)
linear_sum(data, w)
```

markov *Markov Chain and Grey-Markov Prediction Models*

Description

Implements Markov chain prediction for state sequences and the Grey-Markov combined prediction model that integrates GM(1,1) with Markov chain correction.

Usage

```
markov_chain(S, s0, n_steps = 5)

GM11_markov(X, n_ahead = 3, breaks = c(-0.1, -0.05, -0.02, 0, 0.02, 0.05, 0.1))
```

Arguments

S	For markov_chain: State sequence (vector or factor).
s0	For markov_chain: Initial state, must be one of the levels in S.
n_steps	For markov_chain: Number of prediction steps (positive integer, default 5).
X	For GM11_markov: Numeric vector of original time series data.
n_ahead	For GM11_markov: Number of future periods to predict (default 3).
breaks	For GM11_markov: Numeric vector of boundaries for state classification based on relative error. Note: -Inf and Inf are automatically prepended/appended internally.

Details

markov_chain constructs a transition probability matrix from a state sequence and performs multi-step prediction. For states that never appear as a starting state, equal transition probabilities are assigned to maintain row sums of 1. The ultimate stationary distribution is computed via eigenvalue decomposition.

GM11_markov first fits a GM(1,1) model to the original series, then classifies relative errors into states using the specified boundaries, builds a Markov chain model on the error states, and corrects both historical fitted values and future predictions.

Value

markov_chain Returns a list with:

- trans_mat: Transition probability matrix.
- pred_probs: Matrix of state probabilities for each step.
- pred_states: Predicted states (most likely) for each step.
- pi_final: Ultimate stationary distribution.

GM11_markov Returns a data frame with columns:

- Period: Time period labels (T1, T2, ..., T+n).
- Raw: Original values (NA for future periods).
- GM11_fitted: GM(1,1) fitted/predicted values.
- err_state: Relative error state (for historical) or predicted state (for future).
- adj_eff: Adjustment effect (midpoint of state interval).
- Markov_adj: Markov chain adjusted values.

Examples

```
# --- Markov chain prediction ---
# Weather states: Rainy, Cloudy, Sunny
S = factor(c("Sunny", "Sunny", "Cloudy", "Rainy", "Sunny",
             "Cloudy", "Sunny", "Sunny", "Rainy", "Cloudy",
             "Sunny", "Cloudy", "Rainy", "Sunny", "Sunny",
             "Cloudy", "Sunny", "Rainy", "Cloudy", "Sunny"),
           levels = c("Rainy", "Cloudy", "Sunny"))
markov_chain(S, s0 = "Cloudy", n_steps = 3)

# --- Grey-Markov prediction ---
X = c(174, 179, 183, 189, 207, 234, 220.5, 256, 270, 285)
GM11_markov(X, n_ahead = 3)
```

`membership` *Membership Functions for Fuzzy Logic*

Description

A collection of functions to compute membership values for various fuzzy sets, including triangular, trapezoidal, Gaussian, generalized bell, two-parameter Gaussian, sigmoid, difference of sigmoids, product of sigmoids, Z-shaped, PI-shaped, and S-shaped membership functions. Includes a function to visualize membership functions using ggplot2. These are designed for evaluation models in mathematical modeling, compatible with `fuzzy_eval` in the `mathmodels` package.

Usage

```
tri_mf(x, params)

trap_mf(x, params)

gauss_mf(x, params)

gbell_mf(x, params)

gauss2mf(x, params)

sigmoid_mf(x, params)

dsigmoid_mf(x, params)

psigmoid_mf(x, params)

z_mf(x, params)

pi_mf(x, params)

s_mf(x, params)

plot_mf(mf, xlim = c(0, 10), main = NULL)
```

Arguments

x	Numeric vector, input values for which to compute membership.
params	Numeric vector, parameters defining the membership function: • For tri_mf: c(a, b, c), where a <= b <= c (left base, peak, right base). • For trap_mf: c(a, b, c, d), where a <= b <= c <= d (left base, left top, right top, right base). • For gauss_mf: c(sigma, c), where sigma > 0 (spread, center). • For gbell_mf: c(a, b, c), where a > 0, b > 0 (width, shape, center). • For gauss2mf: c(s1, c1, s2, c2), where s1 > 0, s2 > 0 (left spread, left center, right spread, right center). • For sigmoid_mf: c(a, b), where a > 0 (slope, inflection point). • For dsigmoid_mf: c(a1, c1, a2, c2), where a1 > 0, a2 > 0 (slopes and inflection points for two sigmoids). • For psigmoid_mf: c(a1, c1, a2, c2), where a1 > 0, a2 > 0 (slopes and inflection points for two sigmoids). • For z_mf: c(a, b), where a < b (left base, right base). • For pi_mf: c(a, b, c, d), where a < b < c < d (left base, left shoulder, right shoulder, right base). • For s_mf: c(a, b), where a < b (left base, right base).
mf	Function, a membership function with fixed parameters (e.g., function(x) tri_mf(x, c(2, 5, 8))).
xlim	Numeric vector of length 2, x-axis limits for plotting (default c(0, 10)).
main	Character, plot title (default NULL, no title).

Details

These functions support evaluation models in mathematical modeling:

- tri_mf: Triangular membership, linear rise from a to b (peak) and fall to c.
- trap_mf: Trapezoidal membership, linear rise from a to b, plateau from b to c, fall to d.
- gauss_mf: Gaussian membership, bell-shaped curve centered at c with spread sigma.
- gbell_mf: Generalized bell membership, bell-shaped curve with width a, shape b, and center c.
- gauss2mf: Two-parameter Gaussian membership, combining two Gaussians with spreads s1, s2 and centers c1, c2.
- sigmoid_mf: Sigmoid membership, S-shaped curve with slope a and inflection point b.
- dsigmoid_mf: Difference of two sigmoids, combining slopes a1, a2 and inflection points c1, c2.
- psigmoid_mf: Product of two sigmoids, combining slopes a1, a2 and inflection points c1, c2.
- z_mf: Z-shaped membership, decreasing from 1 at a to 0 at b.
- pi_mf: PI-shaped membership, rising from a to b, plateau from b to c, falling to d.
- s_mf: S-shaped membership, increasing from 0 at a to 1 at b.
- plot_mf: Plots a membership function over xlim using ggplot2, suitable for tidyverse workflows.

Membership values can be used to construct fuzzy evaluation matrices for fuzzy_eval. Implemented in base R, except plot_mf, which requires ggplot2.

Value

- For membership functions (`tri_mf`, `trap_mf`, `gauss_mf`, `gbell_mf`, `gauss2mf`, `sigmoid_mf`, `dsigmoid_mf`, `psigmoid_mf`, `z_mf`, `pi_mf`, `s_mf`): A numeric vector of membership values in `[0, 1]`, same length as `x`.
- For `plot_mf`: A ggplot2 object, plotting the membership function.

Examples

```
# Define input values
x = 0:10

# Triangular membership
tri_mf(x, params = c(3, 6, 8))

# Trapezoidal membership
trap_mf(x, params = c(1, 5, 7, 8))

# Gaussian membership
gauss_mf(x, params = c(2, 5))

# Generalized bell membership
gbell_mf(x, params = c(2, 4, 6))

# Two-parameter Gaussian membership
gauss2mf(x, params = c(1, 3, 3, 4))

# Sigmoid membership
sigmoid_mf(x, params = c(2, 4))

# Difference of sigmoids membership
dsigmoid_y = dsigmoid_mf(x, params = c(5, 2, 5, 7))

# Product of sigmoids membership
psigmoid_mf(x, params = c(2, 3, -5, 8))

# Z-shaped membership
z_mf(x, params = c(3, 7))

# PI-shaped membership
pi_mf(x, params = c(1, 4, 5, 10))

# S-shaped membership
s_mf(x, params = c(1, 8))

## Not run:
# Visualize membership functions
plot_mf(\(x) tri_mf(x, c(3, 6, 8)), main = "Triangular MF")
plot_mf(\(x) trap_mf(x, c(1, 5, 7, 8)), main = "Trapezoidal MF")
plot_mf(\(x) gauss_mf(x, c(2, 5)), main = "Gaussian MF")
plot_mf(\(x) gbell_mf(x, c(2, 4, 6)), main = "Generalized Bell MF")
plot_mf(\(x) gauss2mf(x, c(1, 3, 3, 4)), main = "Two-Parameter Gaussian MF")
plot_mf(\(x) sigmoid_mf(x, c(2, 4)), main = "Sigmoid MF")
plot_mf(\(x) dsigmoid_mf(x, c(5, 2, 5, 7)), main = "Difference of Sigmoids MF")
plot_mf(\(x) psigmoid_mf(x, c(2, 3, -5, 8)), main = "Product of Sigmoids MF")
plot_mf(\(x) z_mf(x, c(3, 7)), main = "Z-Shaped MF")
plot_mf(\(x) pi_mf(x, c(1, 4, 5, 10)), main = "PI-Shaped MF")
```

```
plot_mf(\(x) s_mf(x, c(1, 8)), main = "S-Shaped MF")

## End(Not run)
```

`model_logistic` *Logistic Population Growth Model*

Description

Solves the logistic growth equation:

$$\frac{dN}{dt} = r\left(1 - \frac{N}{K}\right)N$$

Usage

```
model_logistic(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(N = 10)`.
`params`	Named numeric vector, e.g. `c(r = 0.5, K = 100)`. **r** Intrinsic growth rate. **K** Carrying capacity.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

The analytical solution is $N(t) = K \,/\, \left(1+(K/N_0-1)\,e^{-rt}\right)$, which can be used to verify numerical accuracy.

Value

A `data.frame` with columns `time` and `N`.

Examples

```
res = model_logistic(
  init   = c(N = 10),
  params = c(r = 0.5, K = 100),
  times  = seq(0, 20, by = 0.1)
)
head(res)
```

`model_lv` *Lotka-Volterra Predator-Prey Model*

Description

Classic two-species predator-prey system:

$$\frac{dx}{dt} = \alpha\, x - \beta\, xy$$

$$\frac{dy}{dt} = \delta\, xy - \mu\, y$$

Usage

```
model_lv(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(x = 40, y = 9)`.
`params`	Named numeric vector with four parameters: **alpha** Prey intrinsic growth rate. **beta** Predation rate (prey killed per predator per unit time). **delta** Predator reproduction rate per prey consumed. **mu** Predator mortality rate. Named `mu` (not `gamma`) to avoid confusion with the recovery rate used in epidemic models.
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

- **x** Prey population.
- **y** Predator population.

Value

A `data.frame` with columns `time`, `x`, `y`.

Examples

```
model_lv(
  init   = c(x = 40, y = 9),
  params = c(alpha = 0.4, beta = 0.04, delta = 0.02, mu = 0.5),
  times  = seq(0, 100, by = 0.1)
)
```

`model_malthus` *Malthusian (Exponential) Growth Model*

Description

Solves the Malthusian growth equation:

$$\frac{dN}{dt} = r\,N$$

Usage

```
model_malthus(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(N = 100)`.
`params`	Named numeric vector, e.g. `c(r = 0.3)`.
	r Intrinsic growth rate (per time unit).
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

The analytical solution is $N(t) = N_0\,e^{rt}$, which can be used to verify numerical accuracy.

Value

A `data.frame` with columns `time` and `N`.

Examples

```
res = model_malthus(
  init   = c(N = 100),
  params = c(r = 0.3),
  times  = seq(0, 10, by = 0.1)
)
# Compare with analytical solution
res$N_exact = 100 * exp(0.3 * res$time)
head(res)
```

`model_seir` *SEIR Epidemic Model*

Description

Four-compartment model that adds an exposed (latent) class:

$$\frac{dS}{dt} = -\beta SI$$
$$\frac{dE}{dt} = \beta SI - \alpha E$$
$$\frac{dI}{dt} = \alpha E - \gamma I$$
$$\frac{dR}{dt} = \gamma I$$

Usage

```
model_seir(init, params, times, ...)
```

Arguments

`init`	Named numeric vector. `R` defaults to 0 if omitted, e.g. `c(S = 980, E = 10, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.3, alpha = 0.2, gamma = 0.1)`. **beta** Transmission rate. **alpha** Rate of progression from exposed to infectious (1/alpha = mean latent period). **gamma** Recovery rate (1/gamma = mean infectious period).
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

- **S** Susceptible — not yet infected.
- **E** Exposed — infected but not yet infectious (mean latent period $1/\alpha$).
- **I** Infectious — actively spreading the pathogen (mean infectious period $1/\gamma$).
- **R** Recovered — permanently immune.

Population size $N = S + E + I + R$ is conserved.

Value

A `data.frame` with columns `time`, `S`, `E`, `I`, `R`.

Examples

```
model_seir(
  init   = c(S = 980, E = 10, I = 10),
  params = c(beta = 0.3, alpha = 0.2, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
```

`model_si` *SI Epidemic Model*

Description

Two-compartment model with no recovery:

$$\frac{dS}{dt} = -\beta SI$$

$$\frac{dI}{dt} = \beta SI$$

Usage

```
model_si(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(S = 990, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.002)`.
	beta Transmission rate (contacts per individual per time).
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

Population size $N = S + I$ is conserved.

Value

A `data.frame` with columns `time`, `S`, `I`.

Examples

```
model_si(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002),
  times  = seq(0, 30, by = 0.1)
)
```

`model_sir` *SIR Epidemic Model*

Description

Three-compartment model with permanent immunity after recovery:

$$\frac{dS}{dt} = -\beta SI$$

$$\frac{dI}{dt} = \beta SI - \gamma I$$

$$\frac{dR}{dt} = \gamma I$$

Usage

```
model_sir(init, params, times, ...)
```

Arguments

`init`	Named numeric vector. `R` defaults to 0 if omitted, e.g. `c(S = 990, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.002, gamma = 0.1)`. **beta** Transmission rate. **gamma** Recovery rate (1/gamma = mean infectious period).
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

Population size $N = S + I + R$ is conserved. The basic reproduction number is $R_0 = \beta N / \gamma$.

Value

A `data.frame` with columns `time`, `S`, `I`, `R`.

Examples

```
model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
```

`model_sis` *SIS Epidemic Model*

Description

Two-compartment model with recovery but no lasting immunity:

$$\frac{dS}{dt} = -\beta SI + \gamma I$$

$$\frac{dI}{dt} = \beta SI - \gamma I$$

Usage

```
model_sis(init, params, times, ...)
```

Arguments

`init`	Named numeric vector, e.g. `c(S = 990, I = 10)`.
`params`	Named numeric vector, e.g. `c(beta = 0.002, gamma = 0.1)`. **beta** Transmission rate. **gamma** Recovery rate (1/gamma = mean infectious period).
`times`	Numeric vector of output times.
`...`	Additional arguments passed to `ode_solver`.

Details

Population size $N = S + I$ is conserved.

Value

A `data.frame` with columns `time`, `S`, `I`.

Examples

```
model_sis(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
```

`ode_solver` *General ODE Solver*

Description

Solves a system of ordinary differential equations (ODEs) using string-formula equations. Equation strings are parsed once at call time and reused across all integration steps, so the solver is substantially faster than approaches that call `parse()` inside the derivative function.

Usage

```
ode_solver(y0, times, equations, params = NULL, method = "lsoda", ...)
```

Arguments

`y0`	A named numeric vector of initial values for all state variables.
`times`	A numeric vector of output times.
`equations`	A named character vector; names are variable names, values are derivative expressions (e.g. `c(S = "-beta*S*I")`).
`params`	A named numeric vector or list of parameters referenced inside `equations`.
`method`	Integration method passed to `deSolve::ode`. Default `"lsoda"`.
`...`	Additional arguments forwarded to `deSolve::ode`.

Details

All equations are parsed with `parse()` exactly once before the integrator starts. A single pre-allocated environment is reused on every derivative evaluation, avoiding repeated memory allocation and garbage collection.

Value

A `data.frame` with a `time` column followed by one column per state variable.

Examples

```
# Built-in wrapper (one-liner)
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
head(sir)

# Custom SEIR model with demography
seir_demo = ode_solver(
  y0    = c(S = 1000, E = 1, I = 0, R = 0),
  times = seq(0, 200, by = 1),
  equations = c(
    S = "mu*N - beta*S*I - mu*S",
    E = "beta*S*I - alpha*E - mu*E",
    I = "alpha*E - gamma*I - mu*I",
```

```
    R = "gamma*I - mu*R"
  ),
  params = c(beta = 0.3, alpha = 0.2, gamma = 0.1, mu = 0.01, N = 1000)
)
tail(seir_demo)
```

---

pca_weight *PCA-Based Weighting Method*

Description

Computes indicator weights using Principal Component Analysis (PCA). The method extracts principal components and uses their variance contribution to derive objective weights for indicators. Optionally handles positive/negative directions of indicators, and supports pre-standardized data.

Usage

```
pca_weight(X, index = NULL, nfs = NULL, varimax = TRUE, method = "abs")
```

Arguments

X	A numeric data frame or matrix where rows represent samples and columns represent indicators.
index	A character vector indicating the direction of each indicator. Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better), and NA for already standardized indicators (no standardization will be applied).
	If `index = NULL` (default), all indicators are treated as `NA`, meaning no standardization is performed.
nfs	Number of principal components to use; by default, all are used.
varimax	Whether to perform Varimax rotation, default is TRUE.
method	Weighting Method. "abs" uses absolute loading values |a_{ji}| (default), "squared" uses a_{ji}^2.

Value

A list containing:

w	Numeric vector of normalized weights for each indicator.
s	Numeric vector of scores for each sample.
lambda	Eigenvalues of principal components (explained variance).
varP	Proportion of variance explained by selected PCs.

Examples

```
# Example: Using PCA to compute indicator weights
ind = c("+","+","-","-")
pca_weight(iris[1:10, 1:4], ind, nfs = 2)
```

plot_compartments *Plot compartment trajectories*

Description

Draws one line per compartment over time. Use `compartments` to select a subset of the state variables; the default is to plot all columns except `time`.

Usage

```
plot_compartments(df, compartments = NULL)
```

Arguments

`df`	A data frame returned by `ode_solver()` or any built-in model function.
`compartments`	Character vector of compartment names to plot. When `NULL` (the default) every column except `time` is plotted.

Value

A ggplot object.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_compartments(sir)
plot_compartments(sir, compartments = c("S", "I"))
```

plot_incidence *Plot daily new infections (dI)*

Description

Computes the daily change in the infectious compartment ΔI directly via `diff()` and plots it as a line chart. The peak is highlighted with a point marker.

Usage

```
plot_incidence(df)
```

Arguments

`df`	A data frame returned by `ode_solver()` or any built-in model function. Must contain columns `time` and `I`.

Value

A ggplot object.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_incidence(sir)
```

---

plot_phase_si *Phase plot S vs I*

Description

Draws a trajectory in the (S, I) plane. Useful for visualising the epidemic orbit and threshold behaviour.

Usage

```
plot_phase_si(df)
```

Arguments

df	A data frame with columns time, S, and I.

Value

A ggplot object.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_phase_si(sir)
```

plot_Rt_estimate *Plot effective reproduction number R_t*

Description

Estimates the time-varying effective reproduction number R_t directly from compartment data. Two methods are available:

`"mechanistic"` Uses $R_t = \beta S(t)/\gamma$. This is the standard formula for an SIR-type model and is recommended when the model dynamics match the SIR structure.

`"normalized"` Uses $R_t = R_0\, S(t)/N$ with $R_0 = \beta N/\gamma$. Suitable when different normalisations are desired.

Usage

```
plot_Rt_estimate(df, params, N = NULL, method = c("mechanistic", "normalized"))
```

Arguments

`df`	A data frame with columns `time` and `S`.
`params`	A named numeric vector or list containing at least `beta` and `gamma`.
`N`	Total population size. If `NULL` (the default), it is estimated from the sum of all state columns at the first time step.
`method`	Estimation method: `"mechanistic"` (default) or `"normalized"`.

Value

A ggplot object with a dashed horizontal line at $R_t = 1$.

Examples

```
sir = model_sir(
  init   = c(S = 990, I = 10),
  params = c(beta = 0.002, gamma = 0.1),
  times  = seq(0, 50, by = 0.1)
)
plot_Rt_estimate(sir, params = c(beta = 0.002, gamma = 0.1))
```

plot_ts *Plot a Time Series*

Description

Renders a clean line chart for one or more time series.

Usage

```
plot_ts(
  x,
  title = "Time Series",
  subtitle = NULL,
  x_lab = "Time",
  y_lab = "Value",
  colour = .PALETTE$observed,
  add_points = FALSE
)
```

Arguments

x	A numeric vector, ts object, tidy tibble with columns (index, value), or a named list of such objects for multi-series.
title	Character. Plot title.
subtitle	Character. Plot subtitle.
x_lab	Character. x-axis label (default "Time").
y_lab	Character. y-axis label (default "Value").
colour	Character. Line colour (ignored for multi-series).
add_points	Logical. Add point markers (default FALSE).

Value

A ggplot object.

Examples

```
data(AirPassengers)
plot_ts(AirPassengers, title = "Air Passengers", y_lab = "Thousands")
```

---

plot_ts_acf *ACF and PACF Plots*

Description

ACF and PACF Plots

Usage

```
plot_ts_acf(
  x,
  max_lag = 40L,
  type = c("acf", "pacf", "both"),
  title = NULL,
  diff = 0L
)
```

Arguments

x	A numeric vector, ts object, or tidy tibble/data.frame with a value column.
max_lag	Integer. Maximum lag (default 40).
type	Character. "acf" (default), "pacf", or "both" (returns a patchwork panel).
title	Character. Plot title.
diff	Integer. Apply diff() this many times before plotting (default 0).

Value

A ggplot or patchwork object.

Examples

```
data(AirPassengers)
plot_ts_acf(log(AirPassengers), type = "both")
```

---

plot_ts_forecast *Plot Historical Series + Forecast with Confidence Bands*

Description

Works with the tibble returned by ts_forecast().

Usage

```
plot_ts_forecast(
  observed,
  forecast_tbl,
  level = c(80, 95),
  title = "Forecast",
  y_lab = "Value",
  trim_n = NULL
)
```

Arguments

observed	A numeric vector or ts object of the historical data.
forecast_tbl	Tibble from ts_forecast().
level	Numeric vector. Which confidence levels to shade (must match columns in forecast_tbl).
title	Character. Plot title.
y_lab	Character. y-axis label.
trim_n	Integer. Show only the last trim_n observations from history (default NULL = show all).

Value

A ggplot object.

Examples

```
data(AirPassengers)
fit = ts_sarima(log(AirPassengers))
fc  = ts_forecast(fit, h = 24)
plot_ts_forecast(log(AirPassengers), fc, title = "Log Air Passengers Forecast")
```

---

plot_ts_garch *GARCH Volatility Plot*

Description

Four-panel overview: (1) observed returns with $\pm 2\sigma$ bands, (2) conditional variance, (3) standardised residuals, (4) histogram of standardised residuals.

Usage

```
plot_ts_garch(garch_result, title = "GARCH Volatility Analysis")
```

Arguments

garch_result	Result list from ts_garch().
title	Character. Plot title.

Value

A patchwork composite ggplot.

Examples

```
r = diff(log(AirPassengers))
res = ts_garch(r)
plot_ts_garch(res)
```

---

plot_ts_pacf *PACF Plot (convenience alias)*

Description

Shortcut for plot_ts_acf(x, type = "pacf").

Usage

```
plot_ts_pacf(x, max_lag = 40L, title = NULL, diff = 0L)
```

Arguments

x	A numeric vector, ts object, or tidy tibble/data.frame with a value column.
max_lag	Integer. Maximum lag (default 40).
title	Character. Plot title.
diff	Integer. Apply diff() this many times before plotting (default 0).

plot_ts_residuals *Residual Diagnostic Plots*

Description

Four-panel diagnostic: (1) residuals over time, (2) ACF of residuals, (3) Q-Q plot, (4) histogram vs normal density.

Usage

```
plot_ts_residuals(model_result, title = "Residual Diagnostics", max_lag = 30L)
```

Arguments

model_result	Result from any ts_*() function that contains a $fitted tibble with a residual column.
title	Character. Plot title.
max_lag	Integer. Maximum lag for ACF (default 30).

Value

A patchwork composite ggplot.

Examples

```
data(AirPassengers)
fit = ts_sarima(log(AirPassengers))
plot_ts_residuals(fit)
```

plot_ts_sarima_garch *Dual-Axis Plot for SARIMA-GARCH: Mean + Conditional Volatility*

Description

Upper panel shows observed values with mean fitted line; lower panel shows the conditional standard deviation from the GARCH component.

Usage

```
plot_ts_sarima_garch(
  sg_result,
  title = "SARIMA-GARCH: Mean & Volatility",
  y_lab = "Value"
)
```

Arguments

sg_result	Result list from ts_sarima_garch().
title	Character. Overall plot title.
y_lab	Character. y-axis label for upper panel.

Value

A patchwork composite ggplot object.

Examples

```
data(AirPassengers)
res = ts_sarima_garch(log(AirPassengers))
plot_ts_sarima_garch(res)
```

---

plot_ts_stl *Plot STL Decomposition Components*

Description

Stacked four-panel plot: observed, trend, seasonal, remainder.

Usage

```
plot_ts_stl(stl_result, title = "STL Decomposition")
```

Arguments

stl_result	Result list from ts_stl().
title	Character. Plot title.

Value

A patchwork composite ggplot.

Examples

```
data(AirPassengers)
res = ts_stl(AirPassengers)
plot_ts_stl(res)
```

---

preprocess *Preprocessing Functions for Data Normalization and Standardization*

Description

A collection of functions to preprocess numeric data, including standardization, L2 norm normalization, Min-Max scaling, centered-type normalization, interval-type normalization, extreme-value-based normalization, initial-value-based normalization, mean-based normalization, and negative-to-positive transformation. These functions transform a numeric vector to a standardized or normalized scale, suitable for various indicator types (positive, negative, centered, interval-based, or extreme-based).

Usage

```
standardize(x, center = TRUE, scale = TRUE)

normalize(x)

rescale(x, type = "+", a = 0, b = 1)

rescale_middle(x, m)

rescale_interval(x, a, b)

rescale_extreme(x, type = "+")

rescale_initial(x, type = "+")

rescale_mean(x)

to_positive(x, type = "minmax")
```

Arguments

x	Numeric vector to be preprocessed.
center	Logical or numeric scalar, passed to `base::scale` for centering (for `standardize`). Default is `TRUE`.
scale	Logical or numeric scalar, passed to `base::scale` for scaling (for `standardize`). Default is `TRUE`.
type	Character scalar specifying the transformation direction or method: **"+"** Positive direction (larger values are better, for `rescale`, `rescale_extreme` and `rescale_initial`). **"-"** Negative direction (smaller values are better, for `rescale` `rescale_extreme` and `rescale_initial`). **"minmax"** Min-max transformation (for `to_positive`). **"reciprocal"** Reciprocal transformation (for `to_positive`).
a	Numeric scalar, lower bound of the output range or optimal interval (for `rescale` and `rescale_interval`).
b	Numeric scalar, upper bound of the output range or optimal interval (for `rescale` and `rescale_interval`).
m	Numeric scalar, optimal value for centered-type normalization (for `rescale_middle`).

Details

These functions support various preprocessing needs in data analysis:

- `standardize`: Applies Z-score standardization (mean = 0, sd = 1), ideal for equalizing variances or normally distributed data.
- `normalize`: Scales the vector to unit length by dividing by its L2 (Euclidean) norm, useful for machine learning or similarity calculations.
- `rescale`: Performs Min-Max scaling to a specified range (default `[0, 1]`), supporting positive or negative indicators.

- `rescale_middle`: Normalizes centered-type indicators, where values closer to an optimal value `m` are better, mapping to `[0, 1]`.
- `rescale_interval`: Normalizes interval-type indicators, where values within `[a, b]` are optimal, mapping to `[0, 1]`.
- `rescale_extreme`: Normalizes using extreme values: `min(x)/x` for positive indicators or `x/max(x)` for negative indicators, often used in grey relational analysis.
- `rescale_initial`: Normalizes by dividing by the first value (`x/x[1]` or `x[1]/x`), commonly used in grey relational analysis.
- `rescale_mean`: Normalizes by dividing by the mean (`x/mean(x)`), commonly used in grey relational analysis.
- `to_positive`: Converts negative indicators to positive using either min-max (`max(x) - x`) or reciprocal (`1/x`) transformation.

Missing values (`NA`) are preserved in the output. For `rescale_initial` and `rescale_mean`, the initial value or mean must be non-zero, respectively.

Value

A numeric vector of the same length as `x`, transformed as follows:

- `standardize`: Standardized values (mean = 0, sd = 1).
- `normalize`: L2 norm normalized values (Euclidean norm, unit length).
- `rescale`: Min-Max scaled values in `[a, b]` (default `[0, 1]`).
- `rescale_middle`: Centered-type normalized values in `[0, 1]`, where 1 indicates `x = m`.
- `rescale_interval`: Interval-type normalized values in `[0, 1]`, where 1 indicates `x` in `[a, b]`.
- `rescale_extreme`: Extreme-based normalized values using `min(x)/x` (positive) or `x/max(x)` (negative).
- `rescale_initial`: Initial-based normalized values using `x/x[1]` or `x[1]/x`.
- `rescale_mean`: Mean-based normalized values using `x/mean(x)`.
- `to_positive`: Transformed values converting negative indicators to positive using min-max or reciprocal transformation.

Examples

```
# Standardization
x = c(4, 1, NA, 5, 8)
standardize(x)

# L2 norm normalization
normalize(x)

# Min-Max normalization (positive direction)
rescale(x)                # Scale to \code{[0, 1]}
rescale(x, type = "-", a = 0.002, b = 0.996)  # Reverse scaling

# Negative-to-positive transformation
to_positive(x)                       # Min-max transformation
to_positive(x, type = "reciprocal")  # Reciprocal transformation

# Centered-type normalization
PH = 6:9
```

```
rescale_middle(PH, 7)

# Interval-type normalization
Temp = c(35.2, 35.8, 36.6, 37.1, 37.8, 38.4)
rescale_interval(Temp, 36, 37)

# Extreme-based normalization
rescale_extreme(x)         # min(x)/x
rescale_extreme(x, "-")    # x/max(x)

# Initial-based normalization
rescale_initial(x)

# Mean-based normalization
rescale_mean(x)
```

`rank_sum_ratio` *Rank Sum Ratio (RSR) Evaluation*

Description

Performs Rank Sum Ratio (RSR) evaluation on a dataset of positive indicators, computing ranks, weighted RSR values, and a linear regression model to fit RSR against probit-transformed ranks. Supports integer or non-integer ranking methods.

Usage

```
rank_sum_ratio(data, w = NULL, method = "int")
```

Arguments

`data`	Data frame with positive indicator data; first column is an ID column for identifying evaluation objects.
`w`	Numeric vector, weights for indicators (default = equal weights).
`method`	Character scalar, ranking method: "int" for integer ranks or "non-int" for scaled ranks in `[1, n]` (default = "int").

Details

The `rank_sum_ratio` function implements the RSR method for evaluating objects based on positive indicators. It ranks the indicators (using integer or non-integer methods), computes weighted RSR values, adjusts ranks with probit transformation, and fits a linear regression model to relate RSR to probit values. The function assumes the first column of `data` is an ID column, and weights (`w`) can be provided or set to equal weights by default.

Value

A list containing:

- `resultTable`: Data frame with RSR values, ranks, cumulative frequencies, probit values, and fitted RSR values.
- `reg`: Linear model object fitting RSR against probit values.
- `rankTable`: Data frame with ranked indicator values.

Examples

```
# Example data
data = data.frame(ID = c("A", "B", "C"), X1 = c(10, 20, 15), X2 = c(5, 10, 8))
w = c(0.4, 0.6)
rank_sum_ratio(data, w, method = "int")
```

---

read_nbs *Read and Combine National Bureau of Statistics XLS Files*

Description

This function reads multiple XLS files downloaded from the National Bureau of Statistics of China website (http://www.stats.gov.cn) without requiring renaming. It extracts the variable name from the "indicator: XXX" text in cell A1 of each file, skips the first 3 rows of headers, reads the first 31 rows of data, and combines the data into a single data frame by stacking vertically based on the indicator names. It also simplifies region names by removing suffixes such as "city", "province", "autonomous region", "clan", or "Uyghur".

Usage

```
read_nbs(paths)
```

Arguments

paths	A character vector containing the file paths to the XLS files.

Value

A tibble containing the combined data with an additional column `indicator` representing the original indicator names and a `region` column with simplified region names.

Examples

```
## Not run:
paths = c("file1.xls", "file2.xls")
data = read_nbs(paths)

## End(Not run)
```

---

regional_economics *Regional Economics Functions*

Description

A collection of functions for calculating regional economics indices, including Location Quotient (LQ), Herfindahl-Hirschman Index (HHI), and Ellison-Glaeser Index (EG). These functions are designed for analyzing regional and industrial data to assess spatial concentration and market structure.

Usage

```
LQ(data, region, total, .cols, .by = NULL)

HHI(x, scaled = FALSE)

EG(data, region, industry, y)
```

Arguments

data	A data frame containing the necessary data for calculations. For LQ, the first row is assumed to be the national total, with the first two columns specifying the region and total, optionally including a grouping column. For EG, it contains region, industry, and indicator columns (e.g., employment or output).
region	The name of the column in data specifying the region (used in LQ and EG).
total	The name of the column in data specifying the total (used in LQ).
.cols	The columns in data for which to calculate the location quotient (used in LQ).
.by	Optional grouping column for LQ, defaults to NULL (no grouping).
x	A numeric vector for calculating the HHI (used in HHI).
scaled	Logical; if TRUE, the HHI is scaled to account for the number of firms. Defaults to FALSE.
industry	The name of the column in data specifying the industry (used in EG).
y	The name of the column in data specifying the indicator (e.g., employment or output, used in EG).

Details

LQ Calculates the Location Quotient for multiple columns with optional grouping. The LQ measures the relative concentration of an industry in a region compared to a national benchmark. The function assumes the first row of data contains national totals, with the first two columns specifying the region and total, and the remaining columns used for LQ calculation.

HHI Calculates the Herfindahl-Hirschman Index, a measure of market concentration based on the squared sum of market shares. If scaled = TRUE, the HHI is normalized to account for the number of firms.

EG Calculates the Ellison-Glaeser Index, which measures the geographic concentration of an industry while controlling for firm size distribution and random distribution effects. It requires data on regions, industries, and an indicator (e.g., employment or output).

Value

LQ A data frame with the region column and calculated location quotients for the specified columns, optionally grouped by .by.

HHI A numeric value representing the HHI, either scaled or unscaled.

EG A tibble with two columns: the industry name and the corresponding EG index value.

Examples

```
# Example data
data = data.frame(
  region = c("National", "Region_A", "Region_B"),
```

```
  total = c(10000, 4000, 6000),
  industry1 = c(2000, 1000, 1000),
  industry2 = c(6000, 2000, 4000)
)

# Calculate Location Quotient
LQ(data, region, total, starts_with("industry"))

# Calculate HHI
x = c(50, 30, 20)
# Calculate the raw HHI
HHI(x)
# Calculate the standard (scaled) HHI
HHI(x, scaled = TRUE)

# Example data for EG
eg_data = data.frame(
 region = c("R1", "R1", "R1", "R1", "R2", "R2", "R3", "R3", "R1", "R2", "R2", "R3"),
 industry = c("A", "A", "A", "A", "A", "A", "A", "A", "B", "B", "B", "B"),
 employment = c(250, 200, 150, 100, 20, 15, 10, 5, 50, 200, 150, 50)
)
EG(eg_data, region, industry, y = employment)
```

`system_evaluation` *System Evaluation Functions for Coupling and Obstacle Analysis*

Description

These functions provide tools for system-level evaluation in multi-indicator systems:

- `coupling_degree()`: Computes coupling degree, coordination index, and coupling coordination degree for subsystems.
- `obstacle_degree()`: Computes obstacle degrees for secondary indicators to identify key constraints in the system, enabling batch processing with tidyverse for grouping and summarization.

Usage

```
coupling_degree(data, w = NULL, id_cols = NULL, type = "standard")

obstacle_degree(data, w = NULL, id_cols = NULL, scaled = FALSE)
```

Arguments

`data`	A data frame with normalized scores (usually in `[0, 1]`) as columns.
`w`	Optional vector of weights for indicators or subsystems; defaults to equal weights if NULL.
`id_cols`	Optional character vector of column names in `data` to preserve as identifiers (not used in calculations).

type	Either "standard" for the standard coupling formula (results concentrated near 1) or "adjusted" for the revised formula (results more uniformly distributed in [0, 1]), as proposed by Wang Shujia, Kong Wei, et al. in "Misconceptions and Corrections of Domestic Coupling Coordination Degree Models, Journal of Natural Resources, 2021, 36(3): 793–810 (In Chinese)"
scaled	Logical. Whether to perform row normalization on obstacle degrees (default FALSE).

Value

A tibble depending on the function:

coupling_degree A tibble with columns:

- ID: Identifier columns specified by id_cols (if provided).
- coupling: Coupling Degree (range 0-1).
- coord: Coordination Index (range 0-1).
- coupling_coord: Coupling Coordination Degree (range 0-1).

obstacle_degree A tibble with:

- ID: Identifier columns specified by id_cols (if provided).
- Columns for secondary indicator obstacle degrees (O_{ij} = (1 - X_{ij}) * w_{ij}).

Suitable for grouping and summarizing (e.g., with tidyverse) to compute primary indicator obstacle degrees (\(U_i \)).

Examples

```
# Sample normalized subsystem scores
df = data.frame(
  ID = LETTERS[1:6],
  s1 = c(0.0162, 0.1782, 0.5490, 0.6730, 0.0207, 0.9875),
  s2 = c(0.2720, 0.6824, 0.0593, 0.4812, 0.8891, 0.5573),
  s3 = c(0.2655, 0.3721, 0.5729, 0.9082, 0.2017, 0.8984)
)
# Coupling Degree Analysis
coupling_degree(df, id_cols = "ID")        # Equal weights
coupling_degree(df, c(0.4, 0.3, 0.3), id_cols = "ID",
                type = "adjusted")         # "adjusted" coupling degree
# Obstacle Degree Analysis
obstacle_degree(df, id_cols = "ID")        # Equal weights
obstacle_degree(df, c(0.4, 0.3, 0.3), id_cols = "ID")
```

---

topsis *TOPSIS Method for Multi-Criteria Decision Making*

Description

Implements the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS) to rank alternatives based on multiple criteria. The function normalizes the decision matrix using L2 norm, applies weights, and computes relative closeness to the ideal solution.

Usage

```
topsis(X, w = NULL, index = NULL)
```

Arguments

X	A numeric matrix or data frame where rows represent alternatives and columns represent criteria.
w	A numeric vector of weights for each criterion. Must be non-negative and sum to 1. If not provided, equal weights are used.
index	A character vector indicating the direction of each indicator: Use "+" for positive indicators (higher is better), "-" for negative indicators (lower is better). If index = NULL (default), all indicators are treated as "+".

Details

The TOPSIS method ranks alternatives by:

1. Normalizing the decision matrix using L2 norm normalization.
2. Applying weights to form a weighted normalized matrix.
3. Identifying positive and negative ideal solutions based on indicator directions.
4. Computing Euclidean distances to ideal solutions.
5. Calculating relative closeness as S0 / (S0 + Sstar), where S0 is the distance to the negative ideal and Sstar is the distance to the positive ideal.

This implementation supports both positive and negative indicators via the index parameter.

Value

A named numeric vector of relative closeness scores (in [0, 1]) for each alternative. Higher values indicate better alternatives. Names are taken from rownames(X) or default to "Sample1", "Sample2", etc.

Examples

```
A = data.frame(
  X1 = c(2, 5, 3),  # "+"
  X2 = c(8, 1, 6)   # "-"
)
w = c(0.6, 0.4)
idx = c("+","-")
topsis(A, w, idx)
```

ts_back_transform *Back-Transform Forecasts to the Original Scale*

Description

Reverses the operations applied by `ts_transform()` to restore point forecasts and confidence intervals to the original scale. Differencing is un-done by cumulative summation using the tail of the original series as the starting level.

Usage

```
ts_back_transform(forecast_tbl, params, x_original)
```

Arguments

`forecast_tbl`	Tibble from `ts_forecast()` (columns: `step`, `forecast`, optionally `lo_*` and `hi_*`).
`params`	The `$params` element from a `ts_transform()` result.
`x_original`	The original (pre-transform) `ts` or numeric vector, needed to supply the last observed level(s) for un-differencing.

Value

The `forecast_tbl` with `forecast` and all interval columns converted to the original scale.

Examples

```
data(AirPassengers)
tr  = ts_transform(AirPassengers, method = "log", diff = 1,
                   seasonal_diff = 1, verbose = FALSE)
fit = ts_sarima(tr$transformed)
fc  = ts_forecast(fit, h = 12)
fc_orig = ts_back_transform(fc, tr$params, AirPassengers)
```

ts_ets *ETS (Error, Trend, Seasonality) Exponential Smoothing*

Description

Wraps `forecast::ets()` and returns a tidy result list.

Usage

```
ts_ets(x, model = "ZZZ", frequency = NULL, ...)
```

Arguments

x	A numeric vector or ts object.
model	Character ETS model string, e.g. "ZZZ" (auto-select, default), "AAN", "AAA".
frequency	Integer. Required when x is a plain numeric vector.
...	Additional arguments forwarded to forecast::ets().

Value

A named list:

model_info One-row tibble: model type, AIC, AICc, BIC, log-likelihood.

parameters Tidy tibble of smoothing parameters and initial states.

fitted Tibble with index, observed, fitted, residual.

model Raw ets object.

Examples

```
data(AirPassengers)
res = ts_ets(AirPassengers)
res$model_info
```

---

ts_forecast *Generate Forecasts from a Fitted Time Series Model*

Description

A unified interface for forecasting from ETS, SARIMA, GARCH, or SARIMA-GARCH model results produced by the ts_*() functions.

Usage

```
ts_forecast(model_result, h = 12, level = c(80, 95), x_ts = NULL)
```

Arguments

model_result	A result list from ts_ets(), ts_sarima(), ts_garch(), or ts_sarima_garch().
h	Integer. Forecast horizon (number of steps ahead).
level	Numeric vector. Confidence levels in percent (default c(80, 95)).
x_ts	Original ts object (required for GARCH/SARIMA-GARCH to preserve time attributes). If NULL, integer steps are used.

Value

A tibble with columns:

step Forecast step (1 to h).

forecast Point forecast.

lo_ Lower bound for each confidence level.

hi_ Upper bound for each confidence level.

sigma Predicted conditional std dev (GARCH models only).

Examples

```
data(AirPassengers)
fit = ts_sarima(log(AirPassengers))
ts_forecast(fit, h = 24)
```

`ts_garch` *GARCH Variance Modeling*

Description

Fits a GARCH(p,q) model (optionally with ARMA mean) using `rugarch::ugarchfit()` and returns tidy results.

Usage

```
ts_garch(
  x,
  garch_order = c(1, 1),
  arma_order = c(0, 0),
  dist = "norm",
  frequency = NULL,
  ...
)
```

Arguments

`x`	A numeric vector or `ts` object (typically returns/residuals).
`garch_order`	Integer vector `c(p, q)`: GARCH order (default `c(1, 1)`).
`arma_order`	Integer vector `c(p, q)`: ARMA order for the mean equation (default `c(0, 0)` = constant mean).
`dist`	Character. Innovation distribution: `"norm"` (default), `"std"` (Student-t), `"ged"`, `"snorm"`, `"sstd"`.
`frequency`	Integer. Required when `x` is a plain vector.
`...`	Additional arguments forwarded to `rugarch::ugarchspec()`.

Value

A named list:

model_info One-row tibble: spec, distribution, log-likelihood, AIC, BIC.

coefficients Tidy coefficient tibble with estimates, std errors, t-stats, p-values.

fitted Tibble: `index`, `observed`, `sigma`, `variance`, `std_residual`.

diagnostics ARCH-LM and Ljung-Box tests on squared residuals.

model Raw `uGARCHfit` object.

Examples

```
set.seed(42)
r = diff(log(AirPassengers))
res = ts_garch(r)
res$model_info
```

`ts_sarima` *SARIMA Model Fitting*

Description

Auto-selects or fits a user-specified SARIMA model via `forecast::auto.arima()` or `forecast::Arima()`, returning a comprehensive tidy result.

Usage

```
ts_sarima(
  x,
  order = NULL,
  seasonal = NULL,
  frequency = NULL,
  stepwise = TRUE,
  approximation = TRUE,
  ...
)
```

Arguments

`x`	A numeric vector or `ts` object.
`order`	Integer vector of length 3: `c(p, d, q)`. `NULL` (default) triggers automatic selection.
`seasonal`	A list `list(order = c(P,D,Q), period = m)` or `NULL` for automatic selection.
`frequency`	Integer. Required when `x` is a plain vector.
`stepwise`	Logical. Use stepwise search in auto.arima (default `TRUE`).
`approximation`	Logical. Use approximation in auto.arima (default `TRUE`).
`...`	Additional arguments forwarded to `forecast::auto.arima()` or `forecast::Arima()`.

Value

A named list:

model_info One-row tibble: ARIMA order string, AIC, AICc, BIC, log-likelihood, sigma².

coefficients Tidy tibble with estimate and std error.

fitted Tibble: `index`, `observed`, `fitted`, `residual`.

diagnostics Ljung-Box test result tibble.

model Raw `Arima` object.

Examples

```
data(AirPassengers)
res = ts_sarima(log(AirPassengers))
res$model_info
```

`ts_sarima_garch` *Two-Stage SARIMA-GARCH Joint Model*

Description

Stage 1: fits a SARIMA model on the level series via `ts_sarima()`. Stage 2: fits a GARCH model on the SARIMA residuals via `ts_garch()`. Returns both sub-models plus a combined fitted tibble.

Usage

```
ts_sarima_garch(
  x,
  sarima_order = NULL,
  sarima_seasonal = NULL,
  garch_order = c(1, 1),
  garch_dist = "std",
  frequency = NULL,
  ...
)
```

Arguments

`x`	A numeric vector or `ts` object.
`sarima_order`	Integer vector `c(p,d,q)` or `NULL` (auto).
`sarima_seasonal`	List `list(order=c(P,D,Q), period=m)` or `NULL`.
`garch_order`	Integer vector `c(p,q)` (default `c(1,1)`).
`garch_dist`	Character. Innovation distribution for GARCH (default `"std"`).
`frequency`	Integer. Required for plain numeric `x`.
`...`	Additional arguments forwarded to `ts_sarima()`.

Value

A named list:

mean_model Result list from `ts_sarima()`.

variance_model Result list from `ts_garch()`.

fitted Combined tibble: `index`, `observed`, `mean_fitted`, `sigma`, `variance`, `std_residual`.

model_info Combined one-row summary tibble.

Examples

```
data(AirPassengers)
res = ts_sarima_garch(log(AirPassengers))
res$model_info
```

ts_stl *STL (Seasonal + Trend + Loess) Decomposition*

Description

Wraps `stats::stl()` and returns both a tidy long-format tibble and the raw `stl` object for further use.

Usage

```
ts_stl(x, frequency = NULL, s_window = "periodic", robust = TRUE, ...)
```

Arguments

`x`	A `ts` object with `frequency > 1`, or a numeric vector with `frequency` specified.
`frequency`	Integer. Required if `x` is a plain numeric vector.
`s_window`	Seasonal smoothing window passed to `stl()`. Use `"periodic"` (default) for purely periodic seasonality.
`robust`	Logical. Use robust fitting in STL (default `TRUE`).
`...`	Additional arguments forwarded to `stats::stl()`.

Value

A named list:

components Tidy `tibble` with columns `index`, `observed`, `trend`, `seasonal`, `remainder`.

strength Tibble: `seasonal_strength` and `trend_strength` (Wang et al. 2006 metrics).

model Raw `stl` object.

Examples

```
data(AirPassengers)
res = ts_stl(AirPassengers)
res$components
```

ts_test *Stationarity Tests for a Time Series*

Description

Runs ADF, KPSS, and PP tests and returns a tidy summary data frame.

Usage

```
ts_test(
  x,
  adf_lags = NULL,
  kpss_type = c("Level", "Trend"),
  pp_type = c("Z(t_alpha)", "Z(alpha)"),
  verbose = TRUE
)
```

Arguments

x	A numeric vector, ts object, or single-column data frame.
adf_lags	Integer. Max lag for ADF (default: NULL = auto via trunc((length(x)-1)^(1/3))).
kpss_type	Character. "Level" (default) or "Trend" for KPSS null hypothesis.
pp_type	Character. "Z(t_alpha)" (default) or "Z(alpha)".
verbose	Logical. Print test results to console (default TRUE).

Value

A tibble with columns:

test Test name (ADF / KPSS / PP)

null_hypothesis Plain-English description of H0

statistic Test statistic

p_value p-value (or NA when only bounds are available)

conclusion Character: "Stationary" or "Non-stationary"

Examples

```
data(AirPassengers)
ts_test(AirPassengers)
```

---

ts_transform *Transform a Time Series for Stationarity*

Description

Applies variance-stabilising transformation (Box-Cox / log) and/or differencing, returning the transformed series together with all parameters needed to back-transform forecasts.

Usage

```
ts_transform(
  x,
  method = c("none", "log", "boxcox"),
  lambda = NULL,
  diff = 0L,
  seasonal_diff = 0L,
  frequency = NULL,
  verbose = TRUE
)
```

Arguments

x	A numeric vector or ts object.
method	Character. Variance transformation: "none" (default), "log", "boxcox" (auto-selects lambda via forecast::BoxCox.lambda()).
lambda	Numeric. Fixed Box-Cox lambda. Ignored unless method = "boxcox". If NULL (default), lambda is estimated.
diff	Integer. Number of regular differences (default 0).
seasonal_diff	Integer. Number of seasonal differences (default 0).
frequency	Integer. Series frequency, required when x is a plain numeric vector and seasonal_diff > 0.
verbose	Logical. Print transformation summary (default TRUE).

Value

A named list:

transformed Transformed ts object ready for modelling.

params Named list with method, lambda, diff, seasonal_diff, frequency — everything needed by ts_back_transform().

summary One-row tibble describing each step applied.

See Also

ts_back_transform

Examples

```
data(AirPassengers)
res = ts_transform(AirPassengers, method = "log", diff = 1,
                   seasonal_diff = 1)
res$summary
plot_ts(res$transformed, title = "Transformed AirPassengers")
```

---

water_quality *Water Quality Dataset*

Description

A dataset containing water quality evaluation metrics for 20 rivers, including dissolved oxygen (O2, positive indicator), pH value (PH, centered indicator), total bacteria count (germ, negative indicator), and plant nutrient content (nutrient, interval indicator with optimal range 10-20). This dataset is suitable for multi-criteria decision analysis, such as weight calculation and fuzzy comprehensive evaluation in the mathmodels package.

Usage

```
water_quality
```

Format

A data frame with 20 rows and 5 columns:

ID Numeric, unique identifier for each river (1 to 20).

O2 Numeric, dissolved oxygen content (mg/L), higher values are better (positive indicator).

PH Numeric, pH value, values closer to 7 are optimal (centered indicator).

germ Numeric, total bacteria count, lower values are better (negative indicator).

nutrient Numeric, plant nutrient content (mg/L), optimal range is 10-20 (interval indicator).

Details

Water Quality Dataset

Source

Simulated data for water quality evaluation, created for demonstration purposes.

Examples

```
# Load the dataset
data(water_quality)

# Preview the data
head(water_quality)
```

Index